SESSION 3

Dışarıya Açılıyoruz

Controller ve Uçtan Uca Akış

Task Manager API • 1.5 Saat

HTTP | REST | Controller | DTO | End-to-End

BAĞLAM

Bu oturum tam olarak nereye oturuyor?

Session 1

Veri Nedir, Nerede Durur?

Entity ile veriyi modelledik.
Repository ile veritabanına eriştik.
H2 ve JPA ile kayıtlar oluştu.

Session 2

Kimin İşi Ne?

Service katmanında iş mantığını topladık. Strategy Pattern ile sıralamayı genişletilebilir yaptık.

Session 3

Dışarıya Açılıyoruz

HTTP kapısını açacağız.
Controller ile endpoint yazacağız.
Uçtan uca akışı göreceğiz.

İlk iki oturum arka planı kurdu; Session 3 o sistemi dış dünyaya açıyor.

HTTP NEDİR?

Backend ile dış dünya arasındaki ortak dil

HyperText Transfer Protocol

İnternette iki tarafın — client ile server'ın — birbiriyle konuşması için kullandığı protokol.

Protokol = iki tarafın da anladığı, üzerinde anlaştığı ortak dil.

Client istek gönderir → Server işler → Cevap döner.
Bu request-response döngüsü HTTP'nin temelidir.

HTTP ile kim konuşur?

Tarayıcı	Chrome, Firefox...
Mobil Uygulama	iOS, Android app
Postman	API test aracı
Başka Servisler	Mikroservisler arası

Request / Response Döngüsü

Peki HTTP tek yol mu? Hayır, ama en yaygını.

gRPC	Hızlı, tip-güvenli RPC. Mikroservislerde.	*Servisler arası iletişim*
WebSocket	Sürekli açık bağlantı, iki yönlü.	*Chat, canlı bildirimler*
Message Queue	Kafka, RabbitMQ ile asenkron.	*Event-driven sistemler*
HTTP / REST	Yaygın, standart, her platform destekler.	*Web & mobil API'ler*

HTTP TEMELİ

HTTP isteğinin dört temel parçası

```
POST
 /api/tasks
Content-Type: application/json

{
  "title": "Spring Boot öğren"
}
```

Method	Ne yapacağız? oku / oluştur / güncelle / sil
URL	Hangi kaynağa erişiyoruz? örn. /api/tasks
Headers	Metadata bilgisi örn. Content-Type
Body	Gönderilen veri POST / PUT / PATCH için

HTTP Metodları

GET	POST	PUT	PATCH	DELETE
Veri okur, değiştirmez	Yeni kayıt oluşturur	Tamamını günceller	Bir kısmını günceller	Kaynağı siler
Görevleri listele	*Yeni görev ekle*	*Tüm alanları değiştir*	*Sadece status'u değiştir*	*Görevi kaldır*

REST TASARIMI

REST: URL kaynağı, method işlemi anlatır

Method	URL	Açıklama
GET	/api/tasks	Tüm görevleri listeler
GET	/api/tasks/5	ID'si 5 olan görevi getirir
POST	/api/tasks	Yeni görev oluşturur
PATCH	/api/tasks/5/complete	Görevi tamamlar
DELETE	/api/tasks/5	Görevi siler

Status Code = Backend'in Kısa Cevabı

200	OK — Her şey yolunda
201	Created — Yeni kaynak oluştu
400	Bad Request — İstek hatalı
404	Not Found — Kayıt bulunamadı
500	Server Error — Sunucu hatası

Neden /tasks/getAll yazmıyoruz?

REST'te URL isimdir (görevler), fiil değil. İşlemi HTTP method belirler. GET = oku, POST = oluştur.

Neden /api prefix'i kullanıyoruz?

Zorunlu değil ama API endpoint'lerini web sayfalarından ayırır. /api/v2/tasks gibi versiyonlama da kolaylaşır.

MİMARİ

Controller katmanda nereye oturuyor?

Kritik Prensip

Controller = trafik polisi. İsteği alır, Service'e iletir, cevabı döner. İş mantığı Controller'da değil, Service'tedir.

SPRING ANNOTATION

@Controller vs @RestController

@Controller

Server-side rendered web uygulamalarında kullanılır.

Method sonucu bir view adı olarak yorumlanır.

Örnek: dashboard.html render etmek.

```
@Controller
public class PageController {
  @GetMapping("/dashboard")
  public String dashboard() {
    return "dashboard";
  }
}
```

@RestController

API yazarken tercih edilir; JSON döndürür.

@Controller + @ResponseBody birleşimi.

Spring, Java objesini Jackson ile JSON'a çevirir.

```
@RestController
@RequestMapping("/api/tasks")
public class TaskController {
  @GetMapping
  public List<Task> getTasks() {
    return taskService.getTasks();
  }
}
```

CONTROLLER

TaskController — Kod İncelemesi

```
@RestController
@RequestMapping("/api/tasks")
@RequiredArgsConstructor
public class TaskController {
  private final TaskService taskService;

  @GetMapping
  public List<Task> getAllTasks(
      @RequestParam(defaultValue="byDate")
      String sort) {
    return taskService.getTasks(sort);
  }

  @GetMapping("/{id}")
  public Task getTask(
      @PathVariable Long id) {
    return taskService.getTaskById(id);
  }
}
```

@RestController
JSON döndüren API controller tanımlar

@RequestMapping
Tüm endpointlere "/api/tasks" prefix ekler

@RequiredArgsConstructor
Final alanlar için otomatik constructor üretir (Lombok)

@GetMapping
HTTP GET isteğini bu metoda bağlar

@RequestParam
URL query parametrelerini (?sort=byTitle) okur

@PathVariable
URL içindeki dinamik değeri (/tasks/5) yakalar

CONTROLLER

POST ve PATCH Endpointleri

Yeni Görev Oluşturma

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Task createTask(
    @RequestBody TaskCreateRequest req) {
  return taskService.createTask(req.title());
}
```

@ResponseStatus(CREATED)
POST başarılı olduğunda 200 yerine 201 döner. Kaynağın oluştuğunu belirtir.

@RequestBody
HTTP gövdesindeki JSON'u Java nesnesine (DTO) otomatik çevirir.

Görevi Tamamlama

```
@PatchMapping("/{id}/complete")
public Task completeTask(
    @PathVariable Long id) {
  return taskService.completeTask(id);
```

DTO (record)

```
record TaskCreateRequest(String title)
{}
```

Java record: otomatik getter, equals, hashCode ve toString üretir.

PARAMETRE BAĞLAMA

Üç temel bağlama mekanizması

@RequestParam

?sort=byTitle gibi query parametrelerini okur

Listeleme, filtreleme, sıralama gibi durumlar

```
GET /api/tasks?sort=byTitle
```

@PathVariable

/tasks/5 gibi URL içindeki dinamik parçayı yakalar

Kaynağın kimliğini URL üzerinden taşır

```
GET /api/tasks/5
```

@RequestBody

HTTP gövdesindeki JSON'u Java nesnesine çevirir

DTO ile birlikte API sözleşmesini netleştirir

```
POST /api/tasks  { "title": "Rapor yaz"
}
```

D T O

Neden Entity'yi doğrudan döndürmüyoruz?

Entity'yi Direkt Döndürmek

Veritabanı modeli API cevabı olmak zorunda değil

İlişkili entity'ler response'u şişirebilir

Recursive JSON sorunu:
`Task → User → Tasks → User → ...`

Güvenlik riski: password hash, internal
flag'ler client'a sızabilir!

DTO ile Çözüm

Sadece ihtiyaç olan alanları döndürür

Frontend ile backend arası net sözleşme

Validation kuralları request seviyesinde

`Temiz response: { id, title, status }`

Java Record ile DTO

```
// Request DTO — sadece gerekli alanları alır
record TaskCreateRequest(String title) {}

// Java record: getter, equals, hashCode, toString otomatik gelir
```

D T O

Kullanmazsak ne olur? Kullanırsak ne olur?

Entity doğrudan döndürülürse

```
GET /api/tasks/1  →  Response:
{
  "id": 1,
  "title": "Rapor yaz",
  "status": "PENDING",
  "createdAt": "2025-03-13T...",
  "user": {
    "id": 42,
    "email": "ali@test.com",
    "passwordHash": "bcrypt$...",
    "tasks": [ ... sonsuz ... ]
  }
}
```

Kırmızı satırlar: gereksiz, tehlikeli, sonsuz döngü riski!

DTO ile kontrollü response

```
GET /api/tasks/1  →  Response:
{
  "id": 1,
  "title": "Rapor yaz",
  "status": "PENDING"
}
```

- Hassas veri sızmaz (password, hash)
- Response boyutu küçük, hızlı
- Frontend ne alacağını bilir
- Recursive JSON riski sıfır

UÇTAN UCA AKIŞ

POST /api/tasks isteği geldiğinde ne olur?

1 — HTTP Request

POST /api/tasks
{ "title": "Rapor yaz" }

→

2 — Controller

JSON → TaskCreateRequest
taskService.createTask() çağrılır

→

3 — Service

Yeni Task oluşturulur
status: PENDING,
createdAt: now()

→

4 — Repository

taskRepository.save(task)
JPA → INSERT SQL üretir

→

5 — Response

Task JSON'a çevrilir
HTTP 201 Created döner

Her katman sadece kendi işini yaptı. Hiçbiri diğer katmanın iç detayını bilmek zorunda kalmadı. Bu ayrışma kod büyüdükçe sistemi yönetilebilir tutar.

HATA YÖNETİMİ

404 ile 500 arasındaki farkı bilinçli yönetmeliyiz

Handler olmadan → 500

```
Status: 500 Internal Server Error
{ "timestamp": "...",
  "status": 500,
  "error": "Internal Server Error" }
```

Handler ile → 404

```
Status: 404 Not Found
{
  "error": "Task bulunamadi: 999"
}
```

GlobalExceptionHandler

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(RuntimeException.class)
  @ResponseStatus(HttpStatus.NOT_FOUND)
  public Map<String, String> handleNotFound(
      RuntimeException ex) {
    return Map.of("error", ex.getMessage());
  }
}
```

Merkezi hata yönetimi
Tüm controller'lardan fırlayan hataları tek yerden yakalar

try-catch gereksiz
Her metoda ayrı ayrı yazmak zorunda kalmazsınız

Genişletilebilir
Validation, auth ve custom exception'lar da eklenebilir

CANLI DEMO

Postman ile test akışı

1	POST	/api/tasks	İki-üç görev oluştur, farklı başlıklar ver	201
2	GET	/api/tasks	Tüm görevleri listele, default sort=byDate	200
3	GET	/api/tasks?sort=byTitle	Strategy Pattern ile farklı sıralama	200
4	GET	/api/tasks/999	Olmayan kayıt → 404 davranışını göster	404
5	PATCH	/api/tasks/1/complete	Görevin DONE olduğunu doğrula	200

"URL'deki tek parametre değişikliğiyle farklı sıralama çalışıyor — Service'e tek satır dokunmadık."

KAPANIŞ

Bu oturumdan ne kalmalı?

- HTTP methodları ve request yapısı netleşti
- REST endpoint tasarımında URL + method + status code üçlüsü oturdu
- Controller'ın görevi: iş mantığı değil, HTTP trafiği yönetmek
- DTO = API sözleşmesi + veri kontrol noktası
- Client → Controller → Service → Repository → Database akışı somutlaştı

"Artık uygulamamız sadece içeride çalışan kod değil; dışarıdan erişilebilen, anlaşılır kurallarla konuşan bir servis."

Sonraki Adımlar

Kısa Vade

Bean Validation, Lombok, DTO genişletme

Orta Vade

Spring Security, JUnit + Mockito, Pagination

Uzun Vade

Docker, CI/CD, Microservice mimarisi

Q&A Başlıkları

/api neden var? • PUT vs PATCH
DTO vs Entity • 404 vs 500